

1. What is Adversarial Search?

- Adversarial search deals with problems where the agent must make decisions in the presence of other agents (adversaries) whose goals conflict with its own.
- Instead of a single-agent environment, we have a multiagent environment with strategic opponents.
- The agent searches for a sequence of actions assuming that other agents also act intelligently.

Key Idea: Choose a move that leads to the best possible outcome, assuming the opponent also plays optimally.

2. Games in AI

- A game is a multiagent decision problem with conflicting goals.
- Typically modeled as:
 - Two (or more) players
 - Competitive (opposing goals)
 - Turn-taking (players move alternately)
 - Often zero-sum (one player's gain is the other's loss).

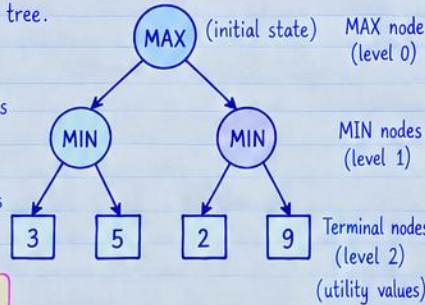
Examples: Chess, Tic-tac-toe, Checkers, Go, Othello, and many strategic games.

3. Game Components

Initial state	: Starting configuration of the game.
Players	: Set of players (e.g., MAX and MIN).
Actions(s)	: Legal moves in a state.
Result(s, a)	: The state that results from taking action a in state s .
Terminal test	: Checks if a state is a terminal (state with no legal moves).
Utility function	: Assigns a numerical value to terminal states (e.g., +1 win, -1 loss, 0 draw for MAX).

4. Game Tree Basics

- A game can be represented as a game tree.
- Nodes represent game states.
- Edges represent actions (moves).
- Levels alternate between the two players (e.g., MAX and MIN).
- Terminal nodes have utility values.
- The tree can be very large (e.g., chess has huge branching factor and depth).



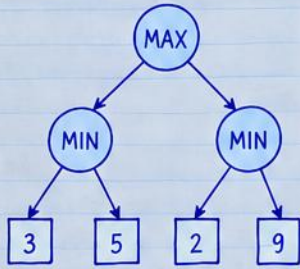
MAX tries to maximize the utility, while MIN tries to minimize it.

5. Minimax Idea

- Assume both players play optimally.
- MAX chooses a move that maximizes the minimum utility it can obtain.
- MIN chooses a move that minimizes the maximum utility it can obtain.
- Propagate utility values from terminals up to the root:
 - At MAX nodes: take the maximum value of the children.
 - At MIN nodes: take the minimum value of the children.

The minimax value at the root gives the utility of the game for MAX, assuming optimal play by both players.

6. Example: Minimax on a Small Game Tree



Step 1: Evaluate terminal nodes (utility values given).

Step 2: Compute MIN nodes:
left MIN = $\min(3, 5) = 3$
right MIN = $\min(2, 9) = 2$

Step 3: Compute root (MAX):
 $\max(3, 2) = 3$

So the minimax value is 3, and MAX will choose the left move.

7. Real-World Examples

Game	Players	Branching Factor (approx)	Typical Depth (approx)	Goal
Chess	Two	35	50+	Checkmate the opponent
Tic-tac-toe	Two	4 (avg)	9	Get 3 in a row (win or draw)
Checkers	Two	7 - 10	40+	Capture all opponent pieces
Go	Two	250 - 300	150+	Control more territory

8. Key Definitions

Game	A multiagent, competitive, turn-taking decision problem.
MAX (Maximizer)	The player who aims to maximize the utility value.
MIN (Minimizer)	The opponent who aims to minimize the utility value.
Game tree	Tree of possible game states, with alternating MAX and MIN levels.
Terminal state	A state with no legal moves (end of game).
Utility function	Assigns a numerical value to terminal states (e.g., +1 win, -1 loss, 0 draw).
Minimax value	The utility of a state assuming optimal play by both players.
Optimal strategy	A strategy (move choice) that guarantees the minimax value.

9. Search vs. Adversarial Search

Aspect	Normal Problem Solving (Single-Agent Search)	Adversarial Search (Multiagent Games)
Environment	Single agent	Multiple agents (opponents)
Goal	Reach a goal state	Maximize utility (win)
Search tree	States and actions	Game tree with alternating MAX and MIN nodes
Terminal test	Goal test	Game over (no legal moves)
Evaluation	Goal reached (success/failure)	Utility value of terminal state
Best action	Any solution path to goal	Move with best minimax value (assuming optimal play)

10. Key Takeaways

- Adversarial search handles multiagent, competitive, turn-taking games.
- Games are modeled using a game tree with MAX and MIN nodes.
- Minimax chooses the move that maximizes the minimum possible utility.
- The utility function defines the value of terminal states.

- Real-world games like chess, checkers and Go are classic examples.
- Adversarial search requires reasoning about an intelligent opponent.
- The minimax algorithm provides an optimal strategy (assuming optimal play).
- Adversarial search is more complex than single-agent search due to the branching factor and depth of game trees.

“In adversarial search, we plan not just for the future, but for an intelligent opponent.”

— AIMA (Ch. 6)

1. Minimax Algorithm (Overview)

- Used for two-player, zero-sum, perfect-information games (e.g., tic-tac-toe, chess).
- Builds a game tree of possible moves.
- Assumes both players play optimally.
- Propagates utility values from terminal states back up to the root.
- Chooses the move with the best guaranteed outcome (maximizes the minimum gain).

Key Idea: Assume the opponent plays optimally and choose the move that maximizes our minimum possible utility.

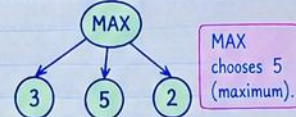
2. Terminal Utilities and Backing Up Values

- Terminal states have predefined utility values (e.g., +1 win, 0 draw, -1 loss).
- These values are backed up the game tree:
- At MAX nodes, take the maximum of child values.
- At MIN nodes, take the minimum of child values.
- This process continues until we compute the value at the root.

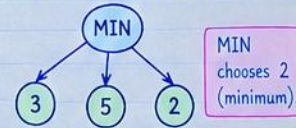
The backed up value at a node represents the final outcome if both players play optimally from that state.

3. MAX and MIN Decisions

- MAX node: chooses the child with the maximum value.



- MIN node: chooses the child with the minimum value.



4. Minimax Algorithm (Step-by-Step)

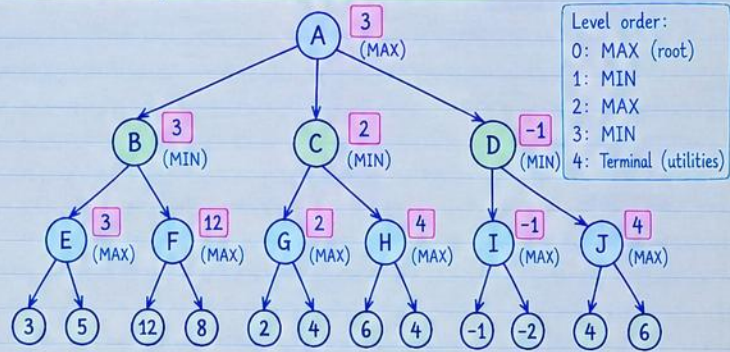
Algorithm: MINIMAX-DECISION(state)

- Use MINIMAX-VALUE(state) to compute value of each successor of the root.
- Return the action with the highest value.

Function: MINIMAX-VALUE(state)

- If state is a terminal state, return its utility.
- If state is a MAX node:
return $\max_{s \in \text{Successors}(\text{state})} \text{MINIMAX-VALUE}(s)$
- If state is a MIN node:
return $\min_{s \in \text{Successors}(\text{state})} \text{MINIMAX-VALUE}(s)$

5. Worked Example: Minimax on a Game Tree



Backing up values (bottom → top):

- $E = \max(3, 5) = 5$
- $F = \max(12, 8) = 12 \rightarrow B = \min(5, 12) = 3$
- $G = \max(2, 4) = 4$
- $H = \max(6, 4) = 6 \rightarrow C = \min(4, 6) = 2$
- $I = \max(-1, -2) = -1$
- $J = \max(4, 6) = 6 \rightarrow D = \min(-1, 6) = -1$
- Finally, $A = \max(3, 2, -1) = 3$

Optimal move:

- At root A (MAX), choose the move to B (value 3).
- If both players play optimally, the game will result in a utility of 3 for MAX.

6. Optimal Play Assumption

- Minimax assumes both players play optimally.
- MAX always tries to maximize the utility.
- MIN always tries to minimize the utility.
- The value returned by minimax is the utility that can be guaranteed if both players play optimally from the current state.

In two-player, zero-sum games, minimax gives the best move assuming an optimal opponent.

7. Complexity of Minimax

- Time complexity: $O(b^m)$
 - b = branching factor (number of moves per state)
 - m = maximum depth of the game tree
 - Minimax explores the entire game tree.
- Space complexity: $O(bm)$
 - Depth-first search with recursion, so space is linear in the depth.

Minimax can be expensive because the number of nodes grows exponentially with the depth.

8. Why Minimax is Correct

- In a game tree, the utility of a state is the outcome that will occur if both players play optimally from that state.
- By choosing the move that maximizes the minimum utility, MAX ensures the best possible outcome against an optimal opponent.
- This can be proved formally using induction on the depth of the game tree.
- Therefore, minimax returns an optimal decision for MAX in two-player, zero-sum, perfect-information games.

9. Advantages and Limitations

Advantages	Limitations
Always finds the optimal move (with full tree search).	Time complexity is exponential ($O(b^m)$).
Simple and easy to understand.	Not feasible for large games (e.g., chess) without pruning.
Works well for small games (e.g., tic-tac-toe, checkers).	Requires complete knowledge of the game (perfect information).
Provides a clear, formal framework for decision making.	Assumes both players play optimally (may not hold in practice).

10. Mini Summary

- Minimax is a search algorithm for adversarial, two-player, zero-sum, perfect-information games.
- Terminal utilities are backed up the game tree.
- MAX chooses maximum child value, MIN chooses minimum child value.
- Assumes optimal play by both players.
- Time complexity: $O(b^m)$, space complexity: $O(bm)$.
- Minimax is correct and gives the optimal move for MAX.
- Useful for small games, but needs improvements (e.g., alpha-beta pruning) for larger games (next pages).

“Minimax chooses the move that maximizes the minimum gain, assuming optimal play by both players.” — AIMA (Ch.6)

1. Depth-Limited Minimax

- Standard minimax works well when terminal states are reachable.
- In many games, the game tree is too large to reach terminal states.
- Depth-limited minimax searches only up to a fixed depth limit l and uses an evaluation function at the cutoff states.
- This allows decision making under limited time and computational resources.

Key Idea: Run minimax to depth limit l and evaluate cutoff states using a heuristic evaluation function instead of terminal utility.

2. Cutoff Test

The search stops and evaluates a state when any of the following is true:

- The state is terminal (win/loss/draw).
- The depth limit l has been reached.
- The available time has run out.
- Other limits (e.g. number of nodes or memory) have been exceeded.

Cutoff Test: terminal(s) OR
depth(s) = l OR
time_limit_reached() OR
node_limit_reached()

3. Evaluation Functions

- Evaluation function $e(s)$ estimates the utility (goodness) of a non-terminal state.
- Needed because we cannot search to terminal states (game trees are huge).
- Provides a heuristic estimate of how good the position is for MAX (or MIN).

Why needed?

- Real games have enormous state spaces (e.g. chess has $\sim 10^{47}$ states).
- Terminal states are too far to reach.
- Allows us to choose good moves at cutoff states using domain knowledge.

4. Properties of a Good Evaluation Function

- Correlates well with the true utility of states.
- Computationally efficient (fast to compute).
- Discriminates between good and bad states.
- Considers multiple features of the game position.
- Adapts to the game (e.g. material, mobility, safety).
- Preferably monotonic: better positions get higher scores as the game progresses.
- Should be consistent with the objective of the game.

5. Horizon Effect

- Occurs when the depth limit is not deep enough to see a significant event (e.g. a forced loss or win) just beyond the horizon.
- The agent may choose a move that looks good at the cutoff depth but is actually bad later.
- Happens because the evaluation function cannot see far enough into the future.

Example:

A move that wins a piece just beyond the depth limit may be missed, so the agent chooses a move that looks good now but leads to a forced loss later.

6. Iterative Deepening in Games

- Perform depth-limited minimax repeatedly with increasing depth limits: 1, 2, 3, ...
- Uses the best result from the deepest completed search depth.
- Helps anytime behavior (can return a move even if time runs out).
- Typically used with a time limit.
- Combines the advantages of depth-limited search and better move ordering from earlier searches.

7. Example: Chess / Checkers

Chess Evaluation Features:

- Material balance (piece values).
- Piece mobility (number of legal moves).
- King safety (castling, protection).
- Pawn structure (isolated, doubled pawns).
- Control of center squares.
- Piece activity and development.
- Endgame knowledge (e.g. king + pawn).

Checkers Evaluation Features:

- Number of pieces.
- Number of kings (more valuable).
- Piece mobility.
- Piece advancement (closer to promotion).
- Board control.
- Avoidance of being captured.

8. Terminal Utility vs. Heuristic Evaluation

Aspect	Terminal Utility	Heuristic Evaluation
Used for	Terminal states only	Non-terminal (cutoff) states
Value type	Exact utility (win/loss/draw)	Estimated utility
Computation	Known, exact	Fast, approximate
Range	Fixed values (e.g. +1, 0, -1)	Any real numbers (e.g. -100 to 100)
Information	Final outcome	Based on features of position
Accuracy	Exact	Approximate (depends on function)
Purpose	Determine game result	Guide search at limited depth

9. Depth-Limited Minimax Algorithm

Algorithm DEPTH-LIMITED-MINIMAX(s, l)

```

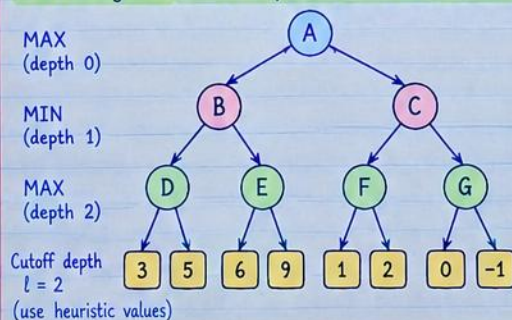
1. if CUTOFF-TEST( $s, l$ ) then
    return EVALUATE( $s$ )
2. if PLAYER( $s$ ) = MAX then
     $v \leftarrow -\infty$ 
    for each action  $a$  in ACTIONS( $s$ ) do
         $v \leftarrow \max(v, \text{DEPTH-LIMITED-MINIMAX}(\text{RESULT}(s, a), l))$ 
    return  $v$ 
3. else // PLAYER( $s$ ) = MIN
     $v \leftarrow +\infty$ 
    for each action  $a$  in ACTIONS( $s$ ) do
         $v \leftarrow \min(v, \text{DEPTH-LIMITED-MINIMAX}(\text{RESULT}(s, a), l))$ 
    return  $v$ 

```

Uses:

- CUTOFF-TEST(s, l)
- EVALUATE(s) (heuristic function)

Tree Diagram: Cutoff Depth and Heuristic Values



Explanation:

- Search is limited to depth $l = 2$.
- States at depth 2 are cutoff states.
- Leaf nodes are evaluated using heuristic function values (shown in yellow).
- Minimax values are backed up from these heuristic values to choose the best move at the root.

10. Exam Tips / Key Points

- Know why evaluation functions are needed.
- Understand cutoff test conditions.
- Be able to write and explain depth-limited minimax algorithm.
- Know the horizon effect and how it occurs.
- Understand iterative deepening in games.
- Know features used in chess/checkers evaluation functions.
- Compare terminal utility vs. heuristic evaluation.
- Practice small tree diagrams with cutoff depth.
- Mention anytime behavior and time limits.
- Emphasize that evaluation functions must be fast and correlate with true utility.

"In adversarial search, we estimate the future when we cannot see it, and search as deep as we can."

— AIMA (Ch.6)

1. Alpha-Beta Pruning Concept

- Alpha-beta pruning is an optimization of minimax search.
- It eliminates branches that cannot possibly affect the final decision.
- It returns the same move as minimax but examines far fewer nodes.
- It prunes parts of the search tree using bounds (α and β) on the best values for MAX and MIN.

Key Idea: If we can prove that a branch cannot influence the final decision, we do not need to explore it.

2. Meaning of α and β

- α (alpha): best value found so far for MAX along the current path. (a lower bound on the final value).
- β (beta): best value found so far for MIN along the current path. (an upper bound on the final value).

$\alpha \leq \text{true minimax value} \leq \beta$
Initially: $\alpha = -\infty$, $\beta = +\infty$

3. Conditions for Pruning

- At a MAX node: if value $\geq \beta$, prune the remaining successors (because MIN will avoid this branch).
- At a MIN node: if value $\leq \alpha$, prune the remaining successors (because MAX will avoid this branch).

In general, prune when $\alpha \geq \beta$ (no need to explore further).

4. Alpha-Beta Algorithm (Recursive)

ALPHA-BETA-SEARCH(state)

return MAX-VALUE(state, $-\infty$, $+\infty$)

MAX-VALUE(state, α , β)

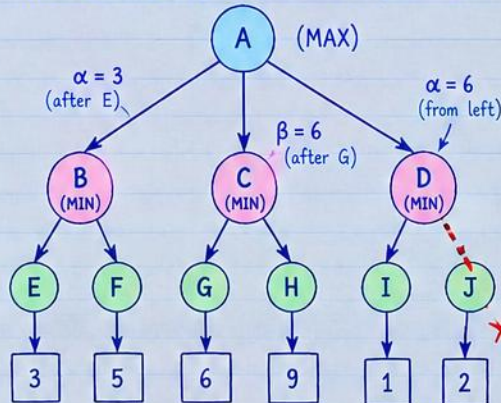
if terminal-test(state) return utility(state)
 $v = -\infty$
for each successor s of state:
 $v = \max(v, \text{MIN-VALUE}(s, \alpha, \beta))$
if $v \geq \beta$ then return v (prune)
 $\alpha = \max(\alpha, v)$
return v

MIN-VALUE(state, α , β)

if terminal-test(state) return utility(state)
 $v = +\infty$
for each successor s of state:
 $v = \min(v, \text{MAX-VALUE}(s, \alpha, \beta))$
if $v \leq \alpha$ then return v (prune)
 $\beta = \min(\beta, v)$
return v

5. Example: Alpha-Beta Pruning on a Game Tree

(values shown during search, left-to-right order)



Search order: left to right

- At B (MIN):
 $\min(3, 5) = 3$
 $\Rightarrow$ A's $\alpha = 3$
- At C (MIN):
 $\min(6, 9) = 6$
 $\Rightarrow$ A's $\alpha = 6$
- At D (MIN):
first child $I = 1$
 $\Rightarrow \beta = 1 \leq \alpha = 6$
 $\Rightarrow$ prune J (do not explore)

pruned!
(since $\beta = 1 \leq \alpha = 6$)

Result: A chooses $\max(3, 6, 1) = 6$ (move to C).
The right branch (J) is pruned, so its value 2 is never examined.

6. Effect of Move Ordering

- The number of nodes pruned depends on the order in which moves are explored.
- Good move ordering (examining best moves first) leads to many more prunings.
- Poor move ordering can lead to little or no pruning (same as minimax).

Move ordering does not affect correctness, only the efficiency.

7. Complexity Improvement

- Plain minimax examines the entire tree:
Time: $O(b^d)$, Space: $O(bd)$
- Alpha-beta pruning can reduce the number of nodes examined, in the best case to $O(b^{d/2})$ (with good move ordering).
- Space complexity remains $O(bd)$ (same as minimax, with depth-first search).

8. Best-Case and Worst-Case

Best Case (perfect move ordering)

- Time complexity: $O(b^{d/2})$
- Searches roughly $b^{d/2}$ nodes.
- Can double the effective search depth.
- Occurs when the best moves are explored first at every node.

Worst Case (poor move ordering)

- Time complexity: $O(b^d)$ (same as minimax)
- No pruning occurs.
- Occurs when the worst moves are explored first.

9. Why Alpha-Beta Returns the Same Move as Minimax

- Alpha-beta pruning never prunes a branch that could contain a better move.
- It only prunes branches that are provably worse than a move already found.
- The bounds (α and β) maintain the same minimax values at each node.
- Therefore, the final move chosen by alpha-beta pruning is always the same as the move chosen by minimax search.

10. Summary / Key Takeaways

- Alpha-beta pruning is an optimization of minimax.
- Uses α (lower bound for MAX) and β (upper bound for MIN).
- Prune when $\alpha \geq \beta$.
- Same move as minimax, but examines fewer nodes.
- Effectiveness depends on move ordering.
- Best case: $O(b^{d/2})$, Worst case: $O(b^d)$.
- Space complexity: $O(bd)$.
- Widely used in game playing (e.g., chess).

"Alpha-beta pruning lets us look further ahead by searching smarter, not harder."

— AIMA (Ch.6)

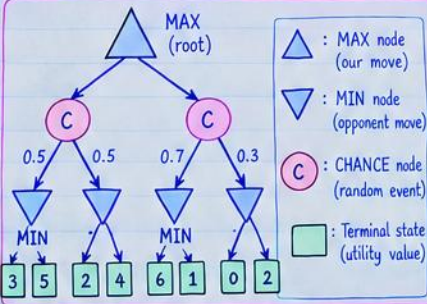
1. Stochastic games and expectiminimax

- Stochastic games include randomness in the game result (e.g., dice rolls, card draws).
- Expectiminimax extends minimax to handle chance nodes by using expected utility values.
- The game tree contains three types of nodes: MAX (our move), MIN (opponent move), and CHANCE (random event).
- At chance nodes, we compute the expected value over all possible outcomes.
- We assume the player maximizes expected utility (e.g., expected win probability or score).

Key Idea: Expectiminimax chooses the action that maximizes the expected utility, considering random events in the game.

2. Chance nodes

- A chance node represents a random event with multiple possible outcomes.
- Each outcome has a known probability.
- The value of a chance node is the weighted average (expected value) of its successor nodes.
- Chance nodes are neither MAX nor MIN; they model the environment's randomness.



3. Expected value idea

- At a chance node, the value is the expected utility of its successors.

Expected value (chance node):

$$V(s) = \sum_{i=1}^k P(s_i) \times V(s_i)$$

$P(s_i)$ = probability of outcome i
 $V(s_i)$ = value of successor state s_i

- The MAX and MIN nodes use the same rules as in minimax:

$$\text{MAX: } V(s) = \max_i V(s_i)$$

$$\text{MIN: } V(s) = \min_i V(s_i)$$

4. Games with randomness (e.g., backgammon)

- Many real games include chance events:
 - Backgammon – dice rolls.
 - Card games – random card draws.
 - Games with hidden information (partially observable).
- Expectiminimax is used to choose actions that maximize the expected chance of winning.
- Backgammon was one of the early test domains for stochastic game search and learning (e.g., using temporal-difference learning).

Example: Backgammon

- Each move depends on dice roll (chance node).
- Different moves have different probabilities.
- Good play maximizes expected winning probability.

5. Summary comparison table

Algorithm	Handles randomness?	Depth limit?	Pruning?	Evaluation	Typical use
Minimax	No	Usually no (or with eval)	No	Terminal or evaluation fcn	Deterministic games (e.g., chess)
Depth-limited minimax	No	Yes	No	Evaluation fcn at depth limit	Large state spaces
Alpha-beta pruning	No	Usually no (or with eval)	Yes	Same as minimax	Faster minimax search
Expectiminimax	Yes (with chance nodes)	Usually no (or with eval)	No (specialized pruning exists)	Expected utility of states	Stochastic games (e.g., backgammon)

6. Applications of adversarial search

- Game playing: chess, checkers, Othello, Go.
- Stochastic games: backgammon, card games.
- Resource allocation and multi-agent systems.
- Security and adversarial planning (opponent modeling).
- Automated negotiation and bidding.
- Robotics in competitive environments.
- Any setting with competing agents and sequential decision making.

7. Strengths and weaknesses

Strengths

- Provides a clear, principled framework for optimal play.
- Works for both deterministic and stochastic games (expectiminimax).
- Can be combined with heuristics and learning.
- Performs well in practice for many games (e.g., chess, backgammon).

Weaknesses

- Exponential time and space complexity.
- Exact search is limited to shallow depths in large games.
- Expectiminimax is more expensive than minimax (more nodes to evaluate).
- Does not handle hidden information directly (need other models).
- Performance depends on evaluation function.

8. Key formulas / complexity recap

Minimax value:

$$\text{MAX: } V(s) = \max_i V(s_i)$$

$$\text{MIN: } V(s) = \min_i V(s_i)$$

Expectiminimax chance node:

$$V(s) = \sum_{i=1}^k P(s_i) \times V(s_i)$$

Time complexity (approx):

- Minimax: $O(b^m)$
- Depth-limited minimax: $O(b^m)$
- Alpha-beta: $O(b^{m/2})$ (best case)
- Expectiminimax: $O(b^m)$ (b = branching factor, m = depth)

Space complexity (approx): $O(bm)$ (depth-first search)

9. Common exam questions

- Explain the difference between minimax and expectiminimax.
- Define a chance node and compute its expected value.
- Draw and evaluate a small game tree with MAX, MIN and CHANCE nodes.
- Compare minimax, depth-limited minimax, alpha-beta and expectiminimax (which handles randomness, pruning, etc.).
- Discuss the role of randomness in games like backgammon.
- Given a simple expectiminimax tree, compute the value of the root.
- Discuss strengths and weaknesses of adversarial search.

10. Final chapter 6 key takeaways

- Adversarial search is used for multi-agent, competitive environments.
- Minimax finds optimal play in deterministic, zero-sum games.
- Alpha-beta pruning speeds up minimax without changing the result.
- Expectiminimax extends minimax to handle stochastic games with chance nodes.
- Chance nodes use expected values based on outcome probabilities.
- Backgammon and similar games illustrate search with randomness.
- Despite exponential complexity, adversarial search works well with heuristics and pruning in practice.
- Understanding adversarial search helps us design intelligent agents that can make good decisions in uncertain, competitive settings.